BRAVO PRODUCTION FINDINGS · 03 OF 05 · OBSERVABILITY

What Bravo can actually see

> *Can Bravo query "all loans stuck in survey"? Are activity logs thin and untagged? Can Bravo tell which upstream is failing, or only that upstreams are unreliable? Is there proper monitoring? And is the log the only durable substrate?*

0	**266,767**	**38**	**2.41M**
CAMUNDA SPANS — THE ORCHESTRATION LAYER EMITS NONE	404S A WEEK FROM ONE ENDPOINT, ACROSS 69% OF SURVEYOR SESSIONS	MONITORS MATCH `BPM`. **ZERO** WATCH A PROCESS	FRONT-END ERRORS IN 7 DAYS, INSTRUMENTED ALL ALONG, NEVER READ

Audience Production, on-call, Squad LOS, Squad S&U **Sources** Datadog us5 — APM, logs, monitors, RUM **Read** 2026-09-10

Most of the stated premises were refuted — in both directions

Bravo can query "all loans stuck in survey"	● TRUE IN SQL ONLY	False everywhere an on-call engineer looks. `surveyor_assignment.assignment_status` is a `WHERE` clause, and that is a genuine Bravo advantage over LORA. But **no dashboard, monitor, SLO or APM view expresses it**, the Camunda runtime tables purge at **90 days**, and `Application.lastStage` is `@Transient` — the current stage is not persisted at all. The capability exists; the practice does not.
…therefore stuck loans cannot be grouped at fleet scale	● REFUTED	By a different route than expected. Camunda's engine log lines carry `@activityName`, `@activityId` and `@processDefinitionId`, so failures **can** be ranked per BPMN activity today: `One Obligor Check` 5,583 errors in 48 hours, `PD Model Check NTB` 1,394. **Nobody was doing it.**
Activity logs are thin / untagged	● REFUTED FOR THE ENGINE	Confirmed for everything else — and **the real defect is different.** The ~5% of lines from the Camunda job executor are richly tagged, including the process definition *version*. The other ~95% are Spring stack traces with no process context. And **no log line carries the application or lead id**, so you can group failures by activity but cannot join a failure to a loan. LORA's `@WorkflowID` *is* the loan id.
We can tell which upstream is failing	● TRUE, AND BETTER THAN LORA	25 upstreams resolve cleanly from `okhttp.request` spans by `@peer.hostname`, with status codes. One Feign client per upstream means **attribution is structural**. LORA puts ~300 proxies behind one route and cannot do this at all without an unreleased PR.

And the thesis itself is upside down

Upstream BFI services are unreliable	● REFUTED	20,802 outbound errors in 7 days and **not one 5xx in the top 20 rows**. Every attributable upstream failure is a 4xx: `401` from IAM (8,533), `404` from repeat-order (5,070), `400` from scheduling (3,508). **These are contract and auth defects, not outages.**
Bravo has no proper monitoring	● REFUTED ON COUNT, CONFIRMED ON SU	38 monitors match `bpm:`. Every one watches a container, a JVM, an HTTP surface or a queue. **Zero watch a process instance, a Camunda incident, the job-executor backlog, or `application_error_tracking`.** Eight are in `Alert` right now; six read `No Data` and cannot fire.
Logs are the only durable substrate	● REFUTED, AND INVERTED	Bravo's durable substrate is **PostgreSQL** — 238 entities, `application_status_log` written by a database trigger, reprocess generations chained as rows. Logs are the *least* durable layer here, and Camunda's own runtime history is the one that expires. **This is the opposite of LORA.**
Bravo's front ends are unmonitored	● REFUTED — AND THE LARGEST GIVE-AWAY	All four iOS consumers are RUM-instrumented at `ALL`. The Surveyor Platform emits **1,962,909 errors in 7 days**, of which **748,686 are HTTP 404s**, and the top one hits **54,350 of 79,090 sessions. No alert can see it, because every success-rate monitor filters on `5*`.**

> **Bravo's problem is not that the data is missing. Most of it is there, and in two places it is better than LORA's.**
>
> The problem is that every alert Bravo owns is scoped to the container and the 5xx — so a system that fails by 4xx and by parking a process cannot trigger one.

“All loans stuck in survey” — answerable in SQL, invisible in telemetry

This is the question LORA cannot answer in Temporal at all — zero search attributes, zero query handlers. Bravo can, and the reason is architectural: business state is a **relational aggregate**, so *“which applications are sitting in surveyor assignment and for how long”* is a `WHERE` clause over `surveyor_assignment.assignment_status` (25 values) joined to `application`. **That advantage is real and this document does not diminish it.** Three things bound it.

THE STAGE IS NOT PERSISTED

`Application.lastStage` is `@Transient`; the current position in the flow is recovered from Camunda’s runtime tables. So the SQL answer is *“stuck in this assignment status”*, not *“stuck at this BPMN step”* — and those differ whenever the step is one the assignment tables do not model.

CAMUNDA’S HISTORY PURGES AT 90 DAYS

`historyTimeToLive: P90D`, history level `full`. `act_hi_actinst` carries roughly **50M rows per 90-day window**. After the window, only the trigger-written `application_status_log` can say what happened — which is a status ladder, not a step trace.

NOBODY HAS WRITTEN THE QUERY DOWN

No dashboard widget, no saved query, no monitor and no SLO in the Datadog org expresses “applications stuck at stage X for more than N hours”. The fleet question is answerable by someone who opens a psql session and knows the schema. **It is not answerable by whoever is on call at 02:00.**

> **Grouping stuck loans at fleet scale does work today, and nobody was doing it.**
>
> That is the part of this finding that was simply wrong — see the next two sections.

The orchestration layer emits no spans at all

OPERATION_NAME	SPANS / 7 D	WHAT IT IS
repository.operation	1,412,356	JPA/Hibernate
spring.handler	990,256	REST controllers
okhttp.request	335,731	outbound calls to upstreams
servlet.request	335,010	inbound HTTP
http.request	84,743	client, other
scheduled.call	1,193	`@Scheduled` / ShedLock jobs
jakarta_rs.request	75	—
Camunda spans of any kind	**0**	**no process instance, no service task, no `JavaDelegate`, no job execution**

Seven operation names cover every span `prod-ms-bpm` produced in 7 days. Grouping `spring.handler` by resource confirms it — the top twenty are all controllers and not one is an `*Activity` bean.

THIS IS STRUCTURAL, NOT A SAMPLING OVERSIGHT

All **483 service tasks** bind via `camunda:delegateExpression` and run inside job-executor threads. Those threads are **not started by an HTTP request**, so the Java agent has no trace to attach to, and nothing in the codebase opens one.

> **The 90% of Bravo's work that is the loan moving through the flow produces no trace.**
>
> APM sees the consoles talking to the database and the engine talking to upstreams. It does not see the process.

THE CONTRAST WITH LORA RUNS THE OTHER WAY

LORA's Temporal SDK OTel interceptor emits **one span per activity attempt**, carrying `@temporalWorkflowID` (= the loan id), `resource_name` (= the activity), `@error.type` and the attempt number. That is how LORA found **47 wedged loans in a 7-day window with no code change**. Bravo has no equivalent span — and if it had a wedged-process class it could not be found this way.

Logs are not thin. They are unjoinable.

545,179	525,181	45,775	1.12M
INFO LINES / 7 D	ERROR LINES / 7 D	WARN LINES / 7 D	TOTAL LINES / 7 D

47% of this service's log volume is ERROR. That number is not a reliability measurement; it is a *formatting artefact*, and both halves of that matter. Clustering the error stream gives 124 patterns, and the largest are individual Java stack-trace frames logged as separate lines — `at <pkg>.<class>.<method>` ×1,571, `at org.springframework.…` ×874, `at java.base/…` ×66. A single exception produces dozens of ERROR lines. Worse, **one Camunda job failure produces two lines at two severities** — `ENGINE-14006` at `warn`, then `ENGINE-16004` at `error`, for the same failure.

THE CONSEQUENCE

The error stream **cannot be counted**, and every log-based monitor built on it inherits that.

BUT THE ENGINE LINES ARE RICHLY TAGGED

```
message: ENGINE-16004 Exception while closing command
  context: Error while invoking Ali Cloud for PD Model
  Check Response
status:  error
version: v2.93.53
@activityId:           Activity_PD_Model_Check_NTB
@activityName:         PD Model Check NTB
@processDefinitionId: NDF2W:127:542b74c3-ab85-11f1…
@processInstanceId:    5b477587-acd4-11f1-be7a-5a673f41…
```

A GENUINELY BETTER VERSION DIMENSION

`@processDefinitionId` carries **the process key and its deployed version** (`NDF2W:127`) — better than LORA's task-queue proxy. The Camunda job executor writes MDC context on its own lines.

Per-activity failure ranking. It took one query.

BPMN ACTIVITY	STATUS	LINES / 2 D
One Obligor Check	error	5,583
PD Model Check NTB	error	1,394
KYC Check	warn	1,377
Publish User Information (Register Keycloak)	error	789
PD Model Check NTB	warn	740
Pilot Branch Check	warn	597
One Obligor	error	434
Retrive Pefindo Spouse Profile Activity	error	326
Request Go Live	error	252
Retrive Pefindo Profile Activity	error	244
Get Agreement Number	error	150

Grouped by `@activityName`, `env:prod`, 48 hours. ~4,750 activity-level failures a day, concentrated in one check.

> **The claim “activity logs are thin” is refuted. The claim “nobody is looking” is not.**

TWO REAL DEFECTS THE TAGGING DOES HAVE

ONLY ~5% OF LINES CARRY PROCESS CONTEXT

17,333 lines in 48 hours match `@processInstanceId:*`, against ~319,000 total. Everything from the REST layer — the consoles, the operator endpoints, every stack trace — has no process context at all.

NO LOG LINE CARRIES THE APPLICATION OR LEAD ID

`@applicationId` was probed and is **absent** from the engine MDC. Bravo can tell you *One Obligor Check is failing 2,800 times a day*. It cannot tell you *which 400 customers are affected* without leaving Datadog — and it cannot tell you whether those are 2,800 loans failing once or forty loans failing seventy times, because the `CARDINALITY(trace_id)` test has no analogue here. There are no traces on the engine path.

QUESTION	BRAVO	LORA
Which activity fails most?	yes — `@activityName`	yes — `@ActivityType`
On which process version?	yes — `@processDefinitionId`	partly — `@TaskQueue`
Which loan is this failure on?	no — a Camunda UUID; resolving it needs a database lookup against `application.process_id`, and only inside the 90-day window	yes — `@WorkflowID` *is* the application UUID
How many times has this attempt been retried?	no — no attempt counter on the line	yes — `@Attempt`

Upstream attribution works — and this inverts the LORA finding

Outbound calls carry `@peer.hostname` per Feign/OkHttp client, **one client per upstream**. 25 hosts resolve cleanly, led by `prod-ms-user-iam` (86,066 spans / 7 d), `prod-ms-employee` (64,725), `prod-ms-product` (31,463), `prod-ms-onboarding` (29,200) and `prod-ms-master` (27,954), down to `prod-ms-backoffice` at 32.

THIS IS THE VIEW LORA DOES NOT HAVE

LORA's gateway routes ~300 inner proxies through one endpoint (`POST /proxy/inner/request-v1`), so **21,128 errors a week report as the same resource** — and the fix, PR 1246, is merged but not in any release tag. Bravo gets per-upstream attribution structurally, from having **113 typed Feign clients instead of one generic proxy**. Give the point to Bravo.

THE ERRORS ARE ENTIRELY 4XX

UPSTREAM	STATUS	ERRORS / 7 D	TRACES
prod-ms-user-iam	401	**8,533**	7,964
prod-ms-repeat-order	404	5,070	3,934
prod-ms-scheduling	400	3,508	3,081
prod-ms-agreement	400	926	449
private.prod.bfi.co.id	400	671	500
gateway.bfi.co.id	422	609	610
prod-ms-kyc-proxy	400	580	474
prod-keycloak-headless	404	287	240

…12 more rows, all 4xx, down to 4 spans. **Not one 5xx appears in the top twenty.** Roughly 20,800 outbound errors a week, all client-side.

"UPSTREAM BFI SERVICES ARE UNRELIABLE" IS NOT WHAT THE DATA SAYS

It says **Bravo asks upstreams for things they will not give it.** `401` from the IAM permission endpoint at ~1,200/day is an auth or token-lifetime defect *in Bravo*, confirmed by the log stream (`FeignException$Unauthorized … [GET] /permission/assigned … "code":"INVALID_KEY"` alongside `TokenExpiredException` ×176). `404` from repeat-order at ~720/day is a lookup for something that does not exist.

THE SPAN:TRACE RATIOS SAY FLEET-WIDE, NOT RETRY LOOPS

IAM is 8,533 errors across 7,964 traces — **1.07:1**, i.e. every affected request fails once. LORA's equivalent table had rows at **10,442 spans in 2 traces**.

Bravo's failure mode is many loans failing once. LORA's was two loans failing ten thousand times.

Both are worth fixing and they need completely different fixes: Bravo's is a defect affecting a broad population; LORA's was a retry policy. This is the clearest single illustration in the pack of what "bounded retry" buys and costs.

38 monitors, none of them on the process

CATEGORY	COUNT	EXAMPLES
Container / pod / JVM resource	11	POD & container CPU and memory ×8, `jvm.heap_memory`, `OutOfMemoryError: Metaspace`, faulty-deployment events
HTTP surface	11	success rate, error rate ×2, p90/average latency, Apdex, throughput anomaly, `Latency Get > 500ms`
RabbitMQ failed-declare	6	one per environment: dev, sit, uat, prod, and three Sharia variants
SLO error budget	3	7 d, 30 d, 90 d — all on the same SLO id
Business / upstream log alerts	5	Check Pefindo Error, Negativelist, `BranchAPIClient` error, CNV success rate, Palav callback
Test / scratch	1	`TEST- succes rate bpm`
Process-level	**0**	—

Nothing watches a Camunda incident, a stuck process instance, the job-executor backlog, `application_error_tracking` row growth, or a per-activity failure rate — even though §3 shows the last of those is one query away.

THREE PATHOLOGIES, EACH WITH AN EXACT LORA COUNTERPART

EIGHT MONITORS ARE IN `ALERT` RIGHT NOW

POD CPU and Memory for Squad S&U; both latency monitors; all three SLO error-budget monitors; and the scratch `TEST-` monitor. **A monitor that has been red long enough to be normal is not a monitor.** LORA found the same thing — a wedged-loan alert firing continuously into a chat room since January.

SIX MONITORS READ `NO DATA` AND CANNOT FIRE

All four `prod-sharia-bpm` resource monitors filter `service:prod-sharia-bpm` while the service is registered as **`prod-sharia-bpm-sharia`**, plus two formula monitors that have never evaluated. This is precisely LORA's eight never-evaluated worker monitors, *tag mismatch and all.*

THE ONE BUSINESS MONITOR FILTERS OUT THE ENGINE'S OWN ERROR CLASS

```
logs("service:prod-ms-bpm checkpefindov2
  -status:(warn OR info) -\"ENGINE-16004\"")
  .rollup("count").by("service").last("5m") > 1
```

`ENGINE-16004` is the line that carries `@activityName` and the actual failure reason. It has been excluded, presumably because it was noisy. **The noise was the signal.** LORA's zombie monitor excluded its own worst failure class for the same reason. Two teams, two platforms, the same reflex.

Dashboards. No dashboard in the org mentions `bpm` or `camunda`. Bravo LOS is covered by nine cloned `LOS Weekly Report Latency` dashboards and two success-rate views — all per-service latency and 5xx. The process view is Camunda Cockpit, which is `permitAll()` at the Spring Security layer and is not part of any on-call flow recorded here.

The sharpest test of the estate: five weeks, and nothing reported

§5 shows **38 monitors matching `bpm` and zero on the process.** There is a concrete test of what that costs. Between **2026-05-23 and 2026-06-30**, across **7 sessions over five weeks**, 61 remote-code-execution process definitions were deployed into the production engine — and on **2026-06-11 07:08:51** one ran and captured the pod hostname.

SURFACE THAT COULD HAVE REPORTED IT	WHAT IT HELD
The 38 `bpm` monitors	**nothing** — none is process-level
Dashboards	**nothing** — none mentions `bpm` or `camunda`
APM spans on the engine path	**nothing** — there are none at all
Camunda's own `act_hi_op_log`	**zero entries** for all 61 deployments
`start_user_id_` on the executed instance	**null**

Found by a person reading the database **three months later** for an unrelated workflow analysis.

> **The finding is not that Bravo was attacked. It is that the telemetry contributed nothing to detecting a five-week intrusion in the core service**, and the discovery path was accidental.

AND THIS CUTS THE OTHER WAY TOO

The engine's history tables are **what made the forensics possible.** `act_re_procdef`, `act_re_deployment`, `act_ge_bytearray` and `act_hi_varinst` preserved the payloads, timestamps, session structure **and the captured command output** — three months on, with no APM and no monitoring. That is the durable-substrate thesis working: **poor at alerting, unusually good at reconstruction.** Do not replace the substrate; put a monitor in front of it.

TWO MONITORS CLOSE THIS, AND THEY ARE THE CHEAPEST HERE

One on new `act_re_deployment` rows whose `name_` is outside the deployment convention; one on process starts whose `proc_def_key_` is outside the known key set. Single SQL predicates over tables that already exist. **Either would have fired on 2026-05-23.**

Not an argument that LORA would have caught it — Temporal Cloud is a different deployment model and no equivalent test exists there. This is Bravo's monitoring against Bravo's own risk surface.

The 404 storm nobody is watching

CONSOLE	SESSIONS	VIEWS	ERRORS	ERR/VIEW
Surveyor Platform Prod	79,090	953,115	**1,962,909**	**2.06**
Operation Platform Prod	17,135	96,707	287,646	**2.97**
Underwriting Platform Prod	15,628	205,666	124,422	0.60
Customer Platform DF	2,594	31,412	38,221	1.22
Total, 7 days	**114,447**	**1,286,900**	**2,413,198**	**1.88**

All four consoles instrumented at `rum_event_processing_state: ALL`. For scale: LORA's back office produces 416,201 errors a week at ≈2.8/view. Bravo's four consoles produce **5.8×** that volume, at a comparable per-view rate. **Neither team was reading RUM.**

THE 404S ARE FOUR ENDPOINTS, AND THEY HIT MOST SESSIONS

ENDPOINT	404S / 7 D	SESSIONS
/bpm/v1/partnership-configuration/feature-configuration	**266,767**	**54,350**
/bpm/v2/surveyor-assignment-form-tab/assignment/{id}	157,623	37,359
/bpm/v1/application-document-tracking	154,097	45,916
/bpm/v1/surveyor-assignment-manual-kyc/scoring/assignment/{id}	143,180	31,565
/surveyor/assignment-detail/{id}	21,125	14,754
/surveyor/assignment	14,565	11,287

69%

OF SURVEYOR SESSIONS — 54,350 OF 79,090 — GET A 404 FROM `FEATURE-CONFIGURATION`

That is not a retry loop and not an edge case; **it is the modal experience of using the Surveyor Platform.** And the endpoint is the busiest one in the service: `PartnershipConfigurationController.getByApplicationId` is the top `spring.handler` resource at **73,056 spans in 48 hours**.

WHY NO ALERT FIRED

```
(hits{service:prod-ms-bpm}
 - hits{service:prod-ms-bpm, http.status_code:5*})
  / hits{service:prod-ms-bpm} < 99
```

A 404 is a success by that definition. Bravo's health model is "not 5xx = healthy" — the same category of assumption as treating a running workflow as a well workflow, and wrong in the same way: the system's dominant failure mode is invisible to the thing meant to detect failure.

WHAT THIS DOCUMENT CANNOT SAY

Whether these 404s are **benign** (an optional configuration probed on every page, 404 meaning "no override") or a **defect** is not determinable from telemetry. It has to be read off the controller. But at 266,767 a week across two-thirds of sessions, on the console that generates 28% of Bravo's support tickets, **somebody has to look** — and the geolocation failures on the same console (20,690/week) are the kind of thing that stops a survey being submitted at all.

The durable-substrate thesis, inverted

The thesis offered was *"logs currently serve as the only durable substrate."* For Bravo that is not merely wrong, **it is upside down.**

LAYER	BRAVO	LORA
Business state	**PostgreSQL, 238 entities, 271 tables, indefinite**	ArangoDB document + Temporal history
Status history	**`application_status_log`, written by a database trigger — durable**	append-only field version chain in the document
Prior attempts	**chained `Application` rows via `prevApplication` / `currentIndex` — every reprocess generation is a durable, queryable row**	rewind invalidates fields; the prior generation is reconstructed from Temporal history
Step-level trace	**Camunda `act_hi_*`, purged at 90 days**	Temporal event history, 30-day retention on closed workflows
Spans	**none for the orchestration layer**	30 days, one per activity attempt
Logs	**~1.1M lines/week, 47% ERROR, no loan id**	~7 days, but carry `@WorkflowID` and `@Attempt`

BRAVO'S DURABLE SUBSTRATE IS THE DATABASE

— and it is **the best one either platform has** for answering "what happened to this loan". Every reprocess generation is a row. Nothing has to be replayed. This is the advantage the relational aggregate genuinely delivers, and it is why the pack credits Bravo's crude "new row per generation" over LORA's more principled rollback.

Bravo's business facts are durable and its process facts are not.

At day 91 you can still say what status a loan reached and when. You cannot say which activity failed, how often, or why. LORA is the mirror image: the loan document and its version chain are durable, but the fleet-level business question — "all loans stuck in survey" — cannot be asked at all.

Ten actions, ordered by value over effort

1 Look at the `feature-configuration` 404 · S, DO THIS FIRST

266,767 a week across 69% of surveyor sessions, on the busiest endpoint in the service, on the console generating 28% of Bravo's tickets. Read the controller; decide whether 404 is the intended "no override" response and if so stop the client treating it as an error. If not, this is a live production defect invisible because nothing alerts below 5xx.

2 Add a 4xx clause to the success-rate monitors · S

Every `prod-ms-bpm` health monitor is `5*`-only. Add per-resource 4xx monitors — at minimum `401` from IAM (8,533/week), `404` on the four surveyor endpoints, `400` on scheduling.

Bravo does not fail with 5xx; it fails with 4xx.

3 Build the per-activity failure monitor that already works · S

`@activityName` grouping is live and untouched. A monitor grouped by `@activityName, @processDefinitionId` is a five-minute change and would today read `One Obligor Check` at ~2,800/day. Give it an owner and a runbook entry *before* creating it.

4 Un-exclude `ENGINE-16004` from the Pefindo monitor · S

It is the line carrying the activity name and the failure reason. If the monitor is too noisy without the exclusion, the fix is a threshold or a per-activity group-by — not suppressing the informative half of the signal.

5 Put the application id into the engine MDC · S–M

One MDC key on the job-executor path turns per-activity ranking into per-loan triage and closes the biggest gap in §3. `application.process_id` already links the two; the log line just needs the business key on it.

6 Fix the six `No Data` monitors · S

Four Sharia monitors filter `service:prod-sharia-bpm` against a service registered as `prod-sharia-bpm-sharia`. Retag or delete. Two formula monitors have never evaluated.

7 Triage the eight monitors in `Alert` · S

Either the thresholds are wrong or the service is degraded; both need an answer.

A permanently red monitor trains people to ignore the channel.

8 Trace the job executor · M

A `@WithSpan` on `BaseActivity.execute()`, or the Camunda OTel plugin, gives one span per service task with process key, version, activity and instance.

The single change that would make Bravo's orchestration visible in APM at all

— and it also delivers per-activity latency, which no metric currently reports.

9 Write down the stuck-loan SQL · S

Bravo's real advantage — fleet questions are `WHERE` clauses — is unusable at 02:00 because nobody saved the query. A notebook plus a monitor on the count converts a capability into an operation.

10 Read RUM in the weekly review · S

Four consoles, 2.4M errors a week, instrumented all along, never consulted. Set a per-console error-per-view SLO; today the Surveyor Platform is at 2.06 and nobody could tell if it regressed.

DO NOT CREATE THESE ALERTS WITHOUT AN OWNER AND A RUNBOOK ENTRY

§5 shows Bravo already has 38 monitors, 8 permanently firing, 6 that cannot fire, and one that had its most useful error class filtered out. **Adding a ninth red light to the same chat room reproduces the problem this document is describing.**

30, 60, 90 days

Ten recommendations plus the runbook precondition, phased against **~10 person-days a month** of Platform/SRE time and a slice of Squad S&U.

SEQUENCING RULE

Repair the alert channel before adding anything to it. So recommendations 6 and 7 land in the first 30 days and recommendation 2 does *not* — even though it is the more valuable signal. It arrives in month 2, after the channel is worth alerting into and after the runbook table exists. The one exception is recommendation 3: created in phase 1 **with its runbook entry and owner attached**, because it needs no new plumbing and is the first process-level monitor Bravo will ever have.

SHARED WITH OTHER DOCUMENTS

Recommendation 1 is also **bravo-delivery.md rec 1** and **bravo-cost.md rec 5**; recommendation 2 is also **bravo-testing.md rec 3**; recommendation 10 pairs with **bravo-delivery.md rec 3**. Phased identically everywhere — do them once.

Days 0–30

ONE LIVE DEFECT, AND A WORKING ALERT CHANNEL

1 **Investigate the `feature-configuration` 404.** *S&U · 2 d.* A written answer — 266,767/week across 69% of sessions is explained.

1 **Fix it** — resolve the lookup, or stop the client raising 404 as an error. *S&U · 3 d.* Surveyor error volume falls measurably in RUM.

7 **Triage the eight monitors in `Alert`.** *Platform/SRE · 2 d.* Each fixed, re-thresholded or deleted. **Zero permanently-red monitors.**

6 **Fix the six `No Data` monitors.** *Platform/SRE · 1 d.*

4 **Un-exclude `ENGINE-16004`** from the Pefindo monitor. *Platform/SRE · 0.5 d.*

3 **Build the per-activity failure monitor** — with owner and runbook entry. *Platform/SRE · 1 d.* **Bravo's first process-level alert.**

5 **Put the application id into the engine MDC.** *S&U · 3 d.* Unblocks per-loan triage and makes recs 2 and 8 far more useful.

9 **Write down the stuck-loan SQL** as a saved notebook. *S&U · 2 d.*

Days 31–60

SEE THE ORCHESTRATION, THEN ALERT ON IT

8 **Trace the job executor.** *S&U · 8 d.* One span per service task. The 90% of Bravo's work that is the process becomes visible in APM, with per-activity latency and a `CARDINALITY(trace_id)` loop test.

– **Alert runbook table** — severity, runbook entry and owner for every signal, retrofitted to the surviving old monitors. *Platform/SRE · 2 d.* **This gates the next row.**

2 **Per-endpoint 4xx monitors.** *Platform/SRE · 3 d.* Bravo's actual failure mode is alertable for the first time.

10 **RUM in the weekly review**, with a per-console error-per-view SLO. *Platform/SRE + EM · 2 d.*

Days 61–90

CONSOLIDATE — NO NEW WORK

– Phase 3 is spent on the deploy-safety and test items in **bravo-testing.md**, which consume this document's output: the job-executor spans from rec 8 are what make the two end-to-end process tests *diagnosable* when they fail.

✓ **What to check at day 90:** that the monitors created in phases 1 and 2 have fired, been acted on, and are **still un-muted**. The failure mode this document describes is not missing tooling — it is a firing alert with no owner. Ninety days is long enough to tell whether that changed.

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
4xx in the health model	A 404 storm on the busiest endpoint pages someone in five minutes instead of running for months at 266k/week.
Per-activity monitor	“One Obligor Check is failing 2,800 times a day” is a number on a dashboard, not a query someone happened to run.
Application id in the MDC	On-call goes from a loan number to its activity failures in one Datadog query, the way LORA on-call already can.
Job-executor spans	The 90% of Bravo’s work that is the process becomes visible in APM, with per-activity latency and a `CARDINALITY(trace_id)` test that separates a broad defect from a loop.
Saved stuck-loan query	Bravo’s genuine architectural advantage — SQL over relational state — becomes something the on-call rota uses, not something the architecture merely permits.
RUM in the weekly review	A surveyor console regression is caught by a number instead of by 315 `Release reject` tickets a month.
Monitors triaged	The alert channel means something again — which is the precondition for every other item on this list.

> **Most of the data is already there. What is missing is an alert that is not scoped to the container and the 5xx.**